02 MAR 2026

AADK TASK 3

Feature Implementation Strategy: Conditionals + Nullability Integration

MindMatrix

Feature Definition

Feature: Event RSVP Screen

This feature allows a user to respond to an event invitation. The user can accept or decline, enter number of guests (optional field), and select a meal preference from a dropdown. The logic ensures the Submit button is only enabled when valid conditions are met and handles null or blank inputs safely.

If:

- User accepts AND guests > 0 → Allow submission.
- User declines → Show “Thank you” message.
- Meal preference is not selected → Default to “Standard”.

Data & State Model

Data Fields & Nullability

Field Name	Type	Nullable?	Purpose
guests	Int?	Yes	Number of guests entered
mealPreference	String?	Yes	Selected meal
rsvpStatus	Boolean	No	Accept (true) / Decline (false)

Compose State Variables

```
var guestsInput by remember { mutableStateOf("") }
var mealPreferenceInput by remember { mutableStateOf<String?>(null) }
var rsvpStatus by remember { mutableStateOf(true) } // true = Accept
var message by remember { mutableStateOf("") }
```

Conditions & Transitions

- If guestsInput.isBlank() → guestsCount = null
- Else → convert using toIntOrNull()
- If RSVP accepted:
  guestsCount != null AND guestsCount > 0 → Enable Submit
  Else → Show error
- If RSVP declined:
  Show Thank You message directly
- If mealPreferenceInput is null → default to "Standard"

Pseudo-Code

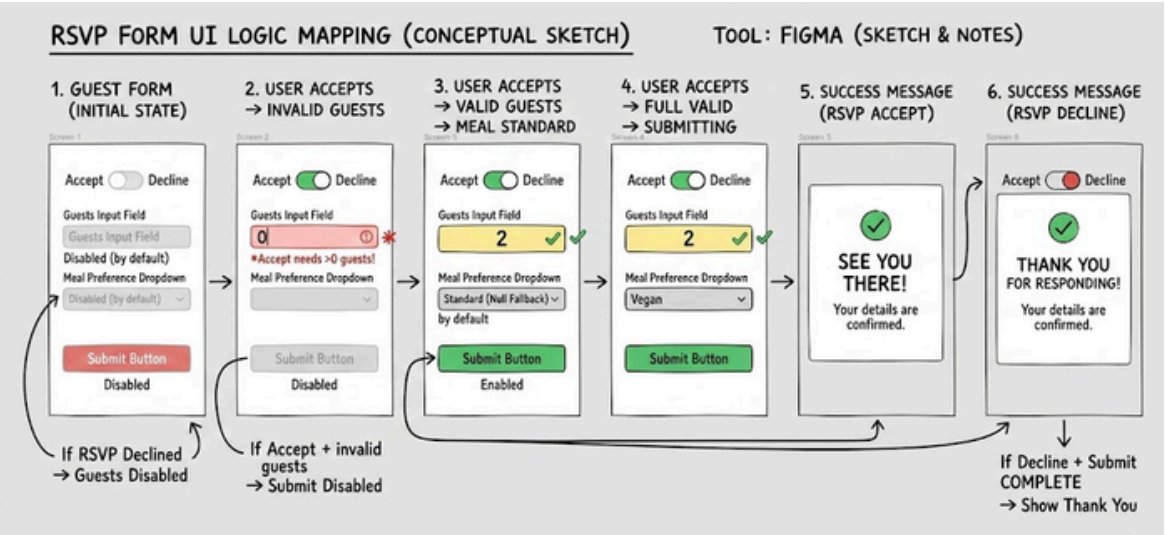
```
val guestsCount = guestsInput.toIntOrNull()
val finalMealPreference = mealPreferenceInput ?: "Standard"
if (rsvpStatus) {  // Accept flow
    if (guestsCount != null && guestsCount > 0) {
        message = "RSVP Confirmed for $guestsCount guests. Meal:
$finalMealPreference"
    } else {
        message = "Please enter valid number of guests"
    }
} else {  // Decline flow
    message = "Thank you for your response!"
}
```

Key Null Safety Used:

- toIntOrNull() to safely convert text input.
- ?: Elvis operator for default meal.
- Explicit null check before proceeding.

UI Sketch & Conditions Mapping

UI Layout Sketch



UI Behavior Mapping

UI Component	Condition	Behavior
Guests Input	RSVP Declined	Disabled
Submit Button	Accept + valid guests	Enabled
Submit Button	Accept + invalid guests	Disabled
Meal Dropdown	Null	Default to "Standard"
Message Text	Decline	Show Thank You

Compose Snippet Example

```
Button(
   onClick = { /* process RSVP */ },
   enabled = if (rsvpStatus) {
      guestsInput.toIntOrNull()?.let { it > 0 } == true
   } else {
      true
   }
) {
   Text("Submit")
}
```

Handling null meal preference:

```
Text(text = mealPreferenceInput ?: "Standard")
```

Edge-Case & Test Planning

Edge Case 1: Guests Field Blank

- guestsInput = ""
- Expected: Show error if Accept selected.
- Logic: toIntOrNull() returns null → Error message shown.

Edge Case 2: Guests = 0

- Expected: Error message (since >0 required).
- Logic: Condition guestsCount > 0 fails.

Edge Case 3: Meal Preference Not Selected & Accept

- Expected: Default meal = "Standard".
- Logic: mealPreferenceInput ?: "Standard" handles it.

Edge Case 4: Rapid Toggle Between Accept/Decline

- Expected:
  Decline → Guests field disabled.
  Accept → Guests field enabled again.
- Logic: UI controlled by rsvpStatus state variable.

Reflection

This feature was more complex because it combines both conditionals and null handling together. I had to think about multiple states at the same time — text input parsing, null values, and UI state changes. Combining conditionals with nullable types increased cognitive load because each branch must also consider null possibilities. To reduce complexity, I used clear default values (like "Standard"), safe parsing (toIntOrNull()), and simple branching instead of deeply nested logic. Planning the state transitions before writing code made everything more structured and easier to understand.